

CS 4372

Assignment #2

Using Trees

Teammates:

Taylor Mitchell

Dylan Ramdhan

Link: https://github.com/dylanramdhan/CS4372/tree/main/Assignment_2

Works Cited

[UCI ML Repo's Wine Quality Database](#)

Scikit Learn [GridSearchCV Reference](#)

Pandas [DataFrame Reference](#)

Seaborn [Pair Plot Reference](#)

Seaborn [Heatmap Reference](#)

1. Plain Decision Tree Classifier

1.1. Before constructing the model, we loaded the Wine Quality dataset into a Pandas DataFrame and examined it for consistency, checking for null values, missing data, and any data inconsistencies (*Fig 1.1.1* and *Fig 1.1.2*).

Features: (6497, 11), Target: (6497, 1), Combined: (6497, 12)

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_sulfur_dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Fig. 1.1.1.

```
fixed_acidity      0
volatile_acidity   0
citric_acid        0
residual_sugar     0
chlorides          0
free_sulfur_dioxide 0
total_sulfur_dioxide 0
density            0
pH                 0
sulphates          0
alcohol            0
quality            0
dtype: int64
```

Fig. 1.1.2.

1.1.1. The dataset contains 6,497 wines (red and white combined), with 11 physicochemical attributes as predictors and quality as the target variable. As shown in *Fig 1.1.2.*, no column contained missing values, so no rows had to be filled or removed. The mean-filling step from our code had no effect, but was kept in case missing values were present.

1.1.2. However, in *Fig 1.1.1.*, it shows that rows 0 and 4 are identical across every attribute, indicating the dataset contains duplicate entries, likely from separate wines producing the same lab measurements.

1.2. After examining the attributes and target variable, we can answer whether or not the attributes are normally distributed, and if not, what could be the reason (*Fig 1.2.1*).

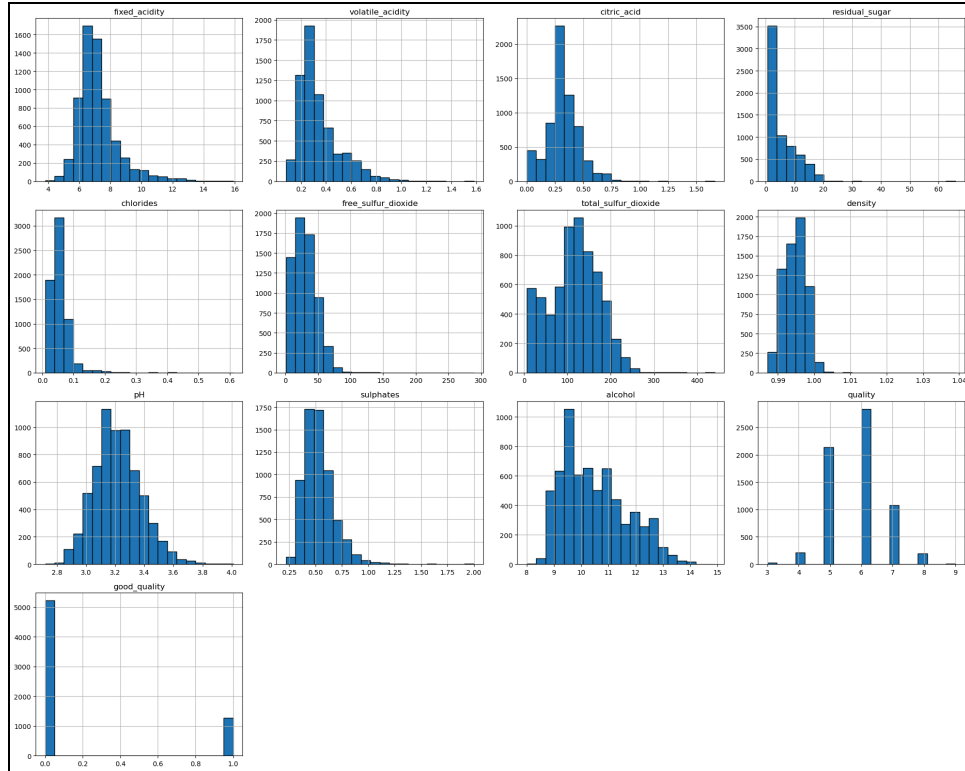


Fig. 1.2.1.

1.2.1. Most attributes are not normally distributed. Only pH comes close to a normal shape, with **fixed_acidity** and **density** roughly bell-shape but right-skewed. Attributes such as **residual_sugar**, **chlorides**, **free_sulfur_dioxide**, and **sulphates** are strongly right-skewed, since these values cannot fall below 0 while a small number of wines contain usually high amounts. **total_sulfur_dioxide** is bimodal because the dataset combines red and white wines, which are produced with different sulfur dioxide levels.

1.2.2. As for the target, **quality** clusters heavily at scores of 5 and 6, since tasters rarely gave extreme scores. After converting it to **good_quality** (quality ≥ 7), the classes are imbalanced at roughly 80% not good to 20% good, which is why metrics beyond accuracy are needed to evaluate the models.

1.3. In order to standardize and normalize the attributes, we used scikit-learn's StandardScaler, converting each attribute to a *mean* of 0 and *standard deviation* of 1, and MinMaxScaler, rescaling each attribute to a range between 0 and 1 (Fig. 1.3.1).

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_sulfur_dioxide	density	pH	sulphates	alcohol
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	0.282257	0.173111	0.191948	0.074283	0.078129	0.102518	0.252868	0.146262	0.386435	0.174870	0.361131
std	0.107143	0.109758	0.087541	0.072972	0.058195	0.061630	0.130235	0.057811	0.124641	0.083599	0.172857
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.214876	0.100000	0.150602	0.018405	0.048173	0.055556	0.163594	0.100829	0.302326	0.117978	0.217391
50%	0.264463	0.140000	0.186747	0.036810	0.063123	0.097222	0.258065	0.149990	0.379845	0.162921	0.333333
75%	0.322314	0.213333	0.234940	0.115031	0.093023	0.138889	0.345622	0.190476	0.465116	0.213483	0.478261
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Fig. 1.3.1.

1.3.1. After normalizing, every attribute ranges from a minimum of 0 to a maximum of 1.

Comparing the mean and median of each attribute also reinforces the skewness found in *Section 1.2*. **residual_sugar** has a mean (*0.074*) roughly double its median (*0.037*), since a few very sweet wine pull the average upward, while **pH** has a nearly equal mean (*0.386*) and median (*0.380*), consistent with its near-normal shape.

1.3.2. Unlike the gradient-based SGDRegressor from Assignment 1, decision trees are not sensitive to feature scale, since each split compares a single attribute against a threshold. Scaling therefore does not change which splits the tree chooses.

1.4. In order to find how the attributes are correlated to each other and to the target variable, we performed a visual analysis using a Seaborn heatmap (*Fig 1.4.1*) and a pair plot key attributes, with each point colored by class (*0 = Not Good*, *1 = Good*; *Fig 1.4.2*).

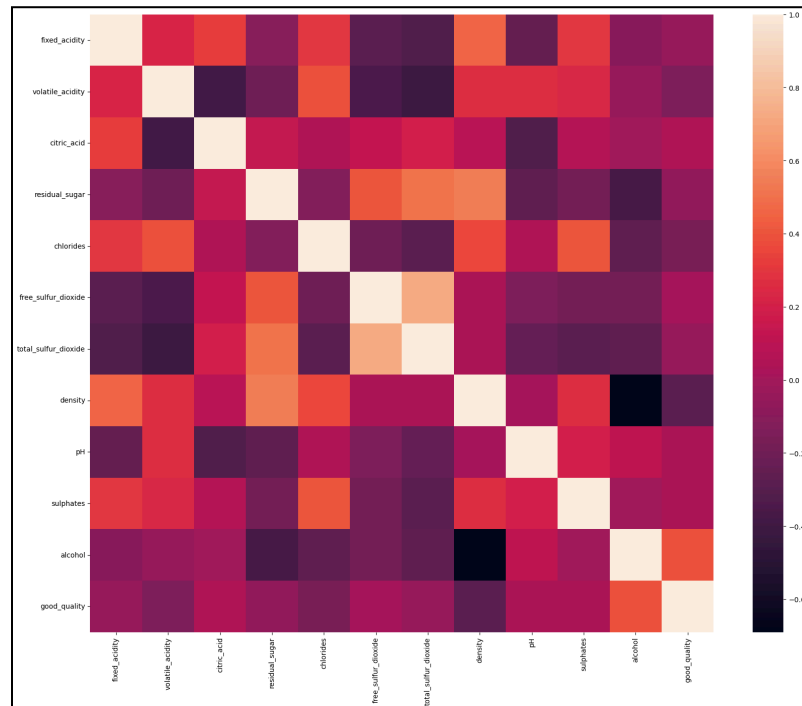


Fig 1.4.1.

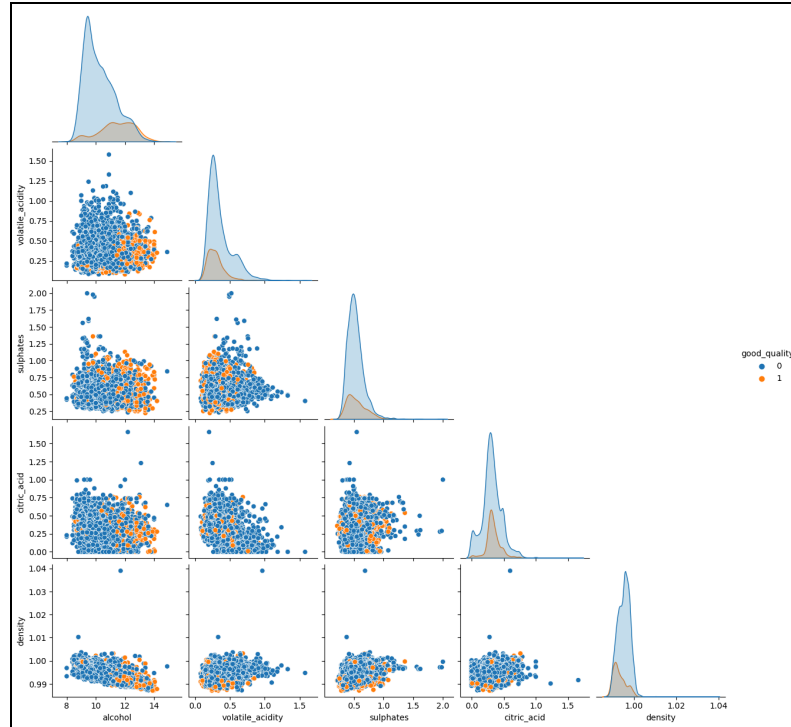


Fig. 1.4.2.

1.4.1. **good_quality** correlates most strongly and positively with **alcohol** (0.39), and negatively with **density** (-0.28), chlorides (-0.16), and **volatile_acidity** (-0.15), meaning higher-alcohol, less dense, and less vinegary wines tend to be rated good. Even the strongest correlation is only moderate, suggesting no single attribute determines quality on its own. Both **density** and **alcohol** are strongly negatively correlated (-0.69), so they carry largely overlapping information.

1.4.2. The pair plot supports that **alcohol** shows to be the clearest separation, with good wines spread across roughly 11% - 13% compared to about 9.5% for not-good wines, while **sulphates** (0.03) and **citric_acid** (0.05) overlap heavily between classes. A single wine with a density near 1.04 also stands out as an outlier.

1.5. This section would be moving into the Model Construction for Decision Tree Classifier. We utilized GridSearchCV with 5-fold cross-validation to tune **criterion**, **max_depth**, and **min_samples_leaf** across 24 combinations. The best parameters (Fig 1.5.1.), test accuracy (Fig. 1.5.2.), and full tuning results (Fig. 1.5.3.).

```
{'dt__criterion': 'gini', 'dt__max_depth': 5, 'dt__min_samples_leaf': 2}
```

Fig. 1.5.1.

0.8146153846153846

Fig. 1.5.2.

	params	mean_test_score	std_test_score	rank_test_score
3	{'dt__criterion': 'gini', 'dt__max_depth': 5, ...}	0.820470	0.007988	1
4	{'dt__criterion': 'gini', 'dt__max_depth': 5, ...}	0.820470	0.007988	1
5	{'dt__criterion': 'gini', 'dt__max_depth': 5, ...}	0.819893	0.007378	3
16	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.819704	0.003418	4
15	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.819704	0.003418	4
17	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.819126	0.003276	6
6	{'dt__criterion': 'gini', 'dt__max_depth': 7, ...}	0.819125	0.013416	7
7	{'dt__criterion': 'gini', 'dt__max_depth': 7, ...}	0.817778	0.013979	8
8	{'dt__criterion': 'gini', 'dt__max_depth': 7, ...}	0.816816	0.011293	9
11	{'dt__criterion': 'gini', 'dt__max_depth': 9, ...}	0.812776	0.005729	10
9	{'dt__criterion': 'gini', 'dt__max_depth': 9, ...}	0.812773	0.012913	11
18	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.812196	0.010911	12
0	{'dt__criterion': 'gini', 'dt__max_depth': 3, ...}	0.811814	0.002329	13
2	{'dt__criterion': 'gini', 'dt__max_depth': 3, ...}	0.811814	0.002329	13
1	{'dt__criterion': 'gini', 'dt__max_depth': 3, ...}	0.811814	0.002329	13
21	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.811619	0.009395	16
19	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.810079	0.012063	17
20	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.809694	0.013123	18
13	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.808927	0.005477	19
14	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.808927	0.005477	19
12	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.808927	0.005477	19
22	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.807385	0.012765	22
10	{'dt__criterion': 'gini', 'dt__max_depth': 9, ...}	0.807193	0.013122	23
23	{'dt__criterion': 'entropy', 'dt__max_depth': ...}	0.804502	0.012957	24

Fig 1.5.3.

1.5.1. The best combination was the gini criterion with a max depth of 5 and minimum of 2 samples per leaf, reaching a cross-validation accuracy of 82% (approx. 0.81). Since a depth of 5 was in the middle of the tested range, we are confident it is the best value rather than a limit of our grid. Shallower trees (depth 3) scored lower from underfitting, while deeper trees (depths 7 and 9) scores lower from over fitting, reflecting the bias-variance trade-off. However, all 24 combinations scored between 0.805 and 0.821, so tuning had only a modest effect.

1.5.2. Test accuracy was 0.815, close to the CV score, so the tree isn't overfitting. However, since about 80% of wines are not good, always guessing "not good" would already score about 80%, so accuracy alone overstates the model's usefulness.

1.6. After the Model Construction, we have used Graphviz to visualize the best estimator tree from GridSearchCV (Fig. 1.6.1.)

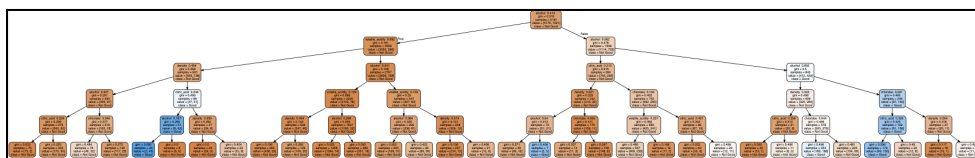


Fig. 1.6.1.

1.6.1. The tree's first split is on **alcohol**, confirming it as the most informative attribute. Wines on the low-alcohol side are about 91% not good, while good wines only become the majority in the highest-alcohol branch (433 good vs. 412 not good). Further splits on **volatile_acidity**, **density**, **chlorides**, and **citric_acid** then refine these groups.

1.7. To evaluate the tuned tree on the test set, we examined its accuracy, classification report, confusion matrix, ROC, and Precision-Recall curves, and features importances (Fig. 1.7.1 - 1.7.6)

	precision	recall	f1-score	support
0	0.84	0.94	0.89	1044
1	0.56	0.29	0.38	256
accuracy			0.81	1300
macro avg	0.70	0.62	0.64	1300
weighted avg	0.79	0.81	0.79	1300

Fig. 1.7.1

Accuracy: 0.8146153846153846

Fig. 1.7.2.

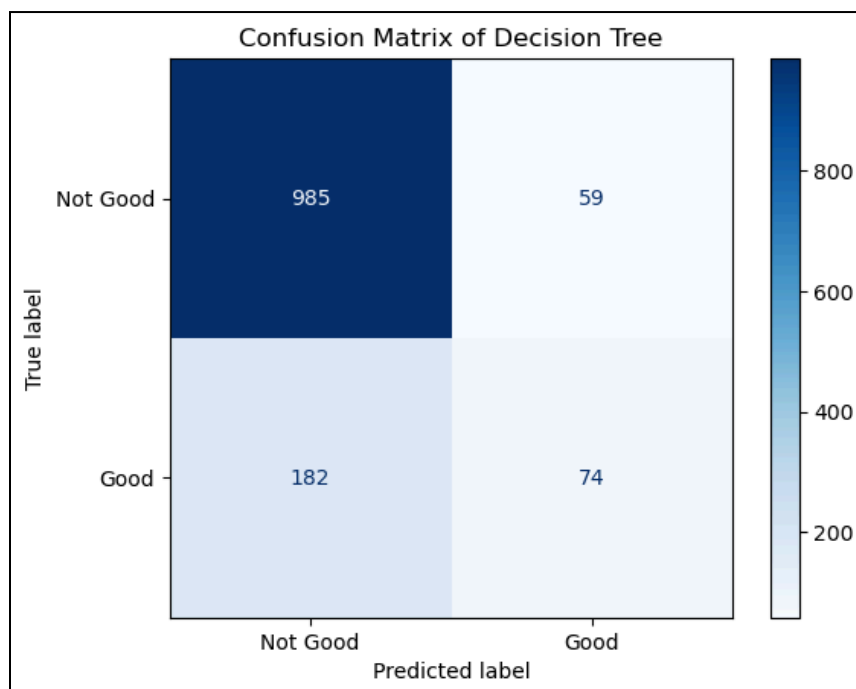


Fig. 1.7.3.

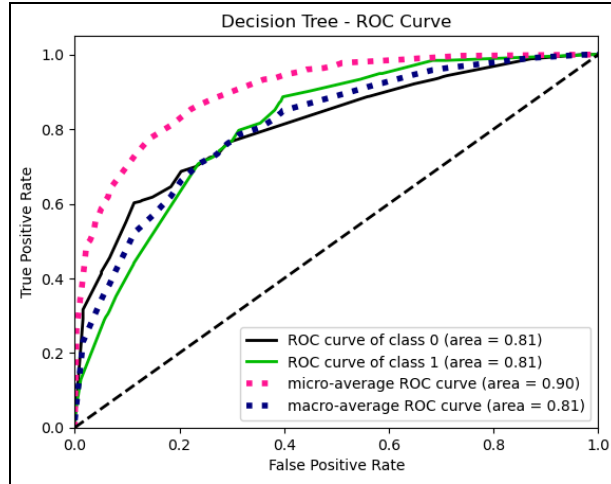


Fig. 1.7.4.

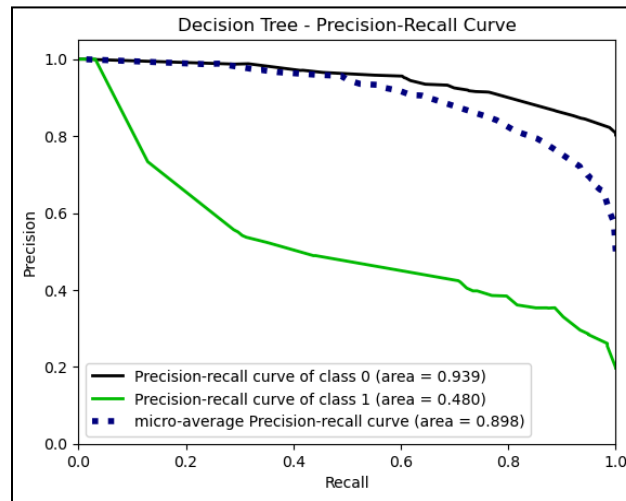


Fig. 1.7.5.

	alcohol	volatile_acidity	density	chlorides	citric_acid
0	0.665263	0.087086	0.085245	0.052318	0.110087

Fig. 1.7.6.

1.7.1. The classification report (Fig. 1.7.1) and confusion matrix (Fig. 1.7.3.) show the tree correctly identified 985 out of 1,044 not-good wines, but only 74 out of 256 good wines. This gives the Good class a recall of 0.29 and precision of 0.56, meaning tree catches fewer than 1 in 3 good wines, and nearly half of its “Good” predictions are wrong, which would result in a lower F1-score of 0.38. The Not Good class, by contrast, scores an F1 of 0.89. This gap comes from the class imbalance: with 4 not-good wines for every good one, the tree learns that predicting “Not Good” is usually safe. As a result, the 0.815 accuracy (Fig. 1.7.2) is only slightly above the 0.80 that could be reached by simply labeling every wine as not good, since 1,044 of the 1,300 test wines are not good.

- 1.7.2. The ROC curve for the Good class has an area of 0.81 (*Fig. 1.7.3*), showing the tree ranks good wines above not-good ones fairly as well. The Precision-Recall curve gives a harsher view, as the area of 0.480 for the Good class compared with 0.939 for Not Good (*Fig. 1.7.4*). Still, 0.480 is more than double the roughly 0.20 a random guess would score, since only 256 of the 1,300 test wines are good, showing the tree is learning useful patterns. Because the Precision-Recall curve focuses only on the rarer good wines, rather than being boosted by the many easy not-good ones, it is the more reliable measure for this dataset.
- 1.7.3. **alcohol** determines the feature importances at 0.665 (*Fig. 1.7.5*), matching its role as the root split. Surprisingly, **citric_acid** ranks second (*0.110*) despite its near-zero correlation with the target, suggesting it helps refine groups already split by alcohol. **density**, contributes little (*0.085*), likely because its information overlaps heavily with alcohol.

2. Random Forest Classifier

2.1. Before constructing the Random Forest model, we loaded the Wine Quality dataset into a Pandas DataFrame and examined it for consistency, checking for null values, missing data, and any data inconsistencies (*Fig. 2.1.1 and 2.1.2*).

Features: (6497, 11), Target: (6497, 1), Combined: (6497, 12)

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_sulfur_dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Fig. 2.1.1.

```
fixed_acidity      0
volatile_acidity   0
citric_acid        0
residual_sugar     0
chlorides          0
free_sulfur_dioxide 0
total_sulfur_dioxide 0
density            0
pH                0
sulphates          0
alcohol            0
quality            0
dtype: int64
```

Fig. 2.1.2.

2.1.1. The dataset contains 6,497 wines (red and white combined), with 11 attributes measured through lab tests (such as acidity, sugar, and alcohol content) as predictors and **quality** as the target variable. As shown in *Fig. 2.1.2.*, no column contained missing values, so no rows had to be filled or removed.

2.1.2. However, *Fig. 2.1.1.* shows that rows 0 and 4 are identical across every attribute, indicating the dataset contains duplicate entries, likely from separate wines producing the same lab measurements.

2.2. After examining the attributes and target variable, we assessed whether the attributes are normally distributed, and if not, what could be the reason (*Fig. 2.2.1.*).

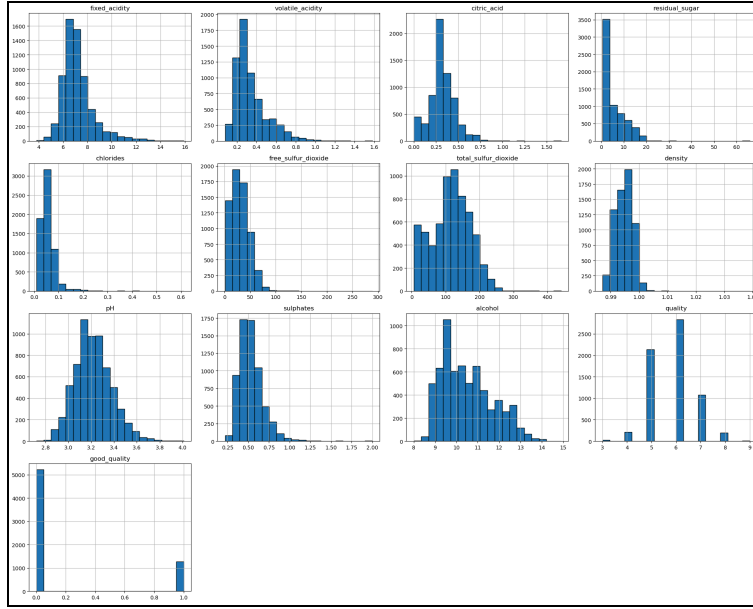


Fig. 2.2.1.

2.2.1. Most attributes are not normally distributed, **pH** is the closest to a bell shape, centered around 3.2, while **fixed_acidity** and **density** are roughly bell-shaped with longer right tails. **Residual_sugar**, **chlorides**, **free_sulfur_dioxide**, and **sulphates** are strongly right-skewed, as these measurements cannot fall below zero while a small number of wines contain unusually high amounts. **total_sulfur_dioxide** shows two peaks because the dataset combines red and white wines, which are produced with different sulfur dioxide levels.

2.2.2. The target **quality** clusters heavily at scores of 5 and 6, since tasters rarely give extreme ratings. After converting it to **good_quality** (quality ≥ 7), roughly 80% of wines are not good and 20% are good. This imbalance matters for the Random Forest, since a model can reach high accuracy just by favoring the majority class, so precision, recall, and the Precision-Recall curve are needed to judge it fairly.

2.3. To standardize and normalize the attributes, we used scikit-learn's **StandardScaler**, converting each attribute to mean of 0 and standard deviation of 1, and **MinMaxScaler**, rescaling each attribute to a range between 0 and 1 (Fig. 2.3.1).

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides	free_sulfur_dioxide	total_sulfur_dioxide	density	pH	sulphates	alcohol
count	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000	6497.000000
mean	0.282257	0.173111	0.191948	0.074283	0.078129	0.102518	0.252868	0.146262	0.386435	0.174870	0.361131
std	0.107143	0.109758	0.087541	0.072972	0.058195	0.061630	0.130235	0.057811	0.124641	0.083599	0.172857
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.214876	0.100000	0.150602	0.018405	0.048173	0.055556	0.163594	0.100829	0.302326	0.117978	0.217391
50%	0.264463	0.140000	0.186747	0.036810	0.063123	0.097222	0.258065	0.149990	0.379845	0.162921	0.333333
75%	0.322314	0.213333	0.234940	0.115031	0.093023	0.138889	0.345622	0.190476	0.465116	0.213483	0.478261
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

Fig. 2.3.1.

2.3.1. After normalizing, every attribute ranges from 0 to 1 across all 6,497 wines. The gap between each attribute's mean and median also reflects its skewness.

residual_sugar has a mean (0.074) about double its median (0.037), as a few very sweet wines pull the average upward, and **chlorides** (0.078) and **free_sulfur_dioxide** (0.103) have similarly low means, with most wines crowded near the bottom of the range. **pH**, in contrast, has a nearly equal mean (0.386) and median (0.380), consistent with its bell-shaped distribution.

2.3.2. A Random Forest is not sensitive to feature scale, since each of its trees splits by comparing a single attribute against a threshold. Rescaling therefore does not change the forest's splits or predictions.

2.4. We examined how the attributes relate to each other and to **good_quality** through a numerical correlation analysis by using a Seaborn heatmap (Fig. 2.4.1) and pair plot colored by class (Fig. 2.4.2)

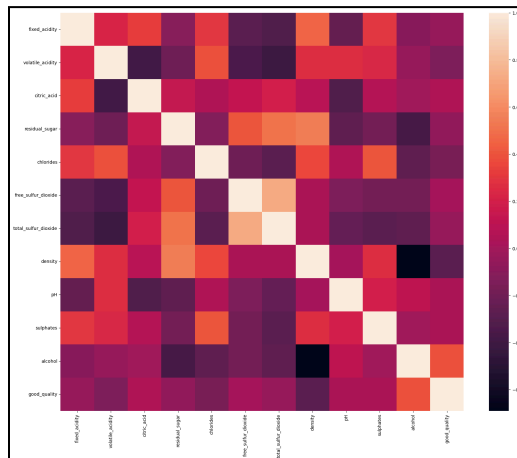


Fig. 2.4.1.

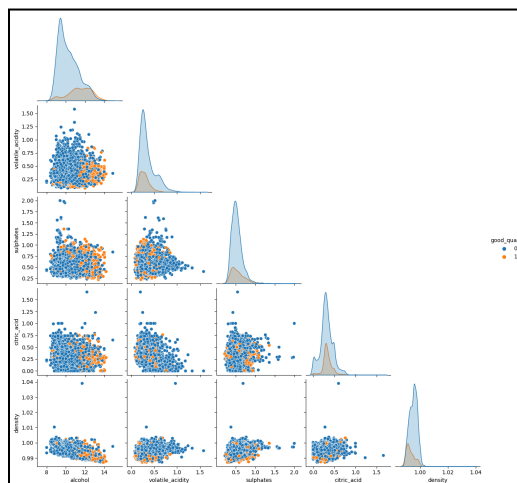


Fig. 2.4.2.

2.4.1. The target's strongest link is with **alcohol** (0.39), the only attribute with a clearly positive relationship. On the negative side, **density** (-0.28) leads, followed by **chlorides** (-0.16) and **volatile_acidity** (-0.15), while the remaining attributes sit near zero. Having no correlation above 0.39, predicting quality requires combining several attributes, which suits a Random Forest, whose many trees consider different attributes.

2.4.2. Among the attributes themselves, **density** and **alcohol** form the darkest square in the heatmap (-0.69), since alcohol is less dense than water. **free_sulfure_dioxide** and **total_sulfur_dioxide** are also strongly linked, as the free amount is part of the total, and **residual_sugar** rises with **density**, since dissolved sugar makes wine heavier. Because the forest limits each split to a random subset of attributes, closely related attributes like density and alcohol may share importance across its trees.

2.4.3. In the pairplot, good wines gather at higher **alcohol** levels, forming the clearest split between classes, while **sulphates** and **citric_acid** show heavily overlapping classes, matching their near-zero correlations (0.03 and 0.05). The plot also reveals a lone wine with a density near 1.04, far outside the rest of the data.

2.5. Using GridSearchCV with 10-fold cross-validation, we tuned **max_depth**, **max_features**, and **n_estimators** across 18 combinations (Fig. 2.5.1 - 2.5.3).

```
{'max_depth': 9, 'max_features': 'sqrt', 'n_estimators': 50}
```

Fig. 2.5.1.

0.8384615384615385

Fig. 2.5.2.

	params	mean_test_score	std_test_score	rank_test_score
15	('max_depth': 9, 'max_features': 'log2', 'n_es...	0.846064	0.005791	1
12	('max_depth': 9, 'max_features': 'sqrt', 'n_es...	0.846064	0.005791	1
13	('max_depth': 9, 'max_features': 'sqrt', 'n_es...	0.843948	0.007577	3
16	('max_depth': 9, 'max_features': 'log2', 'n_es...	0.843948	0.007577	3
17	('max_depth': 9, 'max_features': 'log2', 'n_es...	0.841446	0.007024	5
14	('max_depth': 9, 'max_features': 'sqrt', 'n_es...	0.841446	0.007024	5
7	('max_depth': 7, 'max_features': 'sqrt', 'n_es...	0.838559	0.007837	7
10	('max_depth': 7, 'max_features': 'log2', 'n_es...	0.838559	0.007837	7
6	('max_depth': 7, 'max_features': 'sqrt', 'n_es...	0.838367	0.004430	9
9	('max_depth': 7, 'max_features': 'log2', 'n_es...	0.838367	0.004430	9
11	('max_depth': 7, 'max_features': 'log2', 'n_es...	0.836058	0.008276	11
8	('max_depth': 7, 'max_features': 'sqrt', 'n_es...	0.836058	0.008276	11
2	('max_depth': 5, 'max_features': 'sqrt', 'n_es...	0.825670	0.008547	13
5	('max_depth': 5, 'max_features': 'log2', 'n_es...	0.825670	0.008547	13
4	('max_depth': 5, 'max_features': 'log2', 'n_es...	0.823362	0.009676	15
1	('max_depth': 5, 'max_features': 'sqrt', 'n_es...	0.823362	0.009676	15
3	('max_depth': 5, 'max_features': 'log2', 'n_es...	0.822594	0.009871	17
0	('max_depth': 5, 'max_features': 'sqrt', 'n_es...	0.822594	0.009871	17

Fig. 2.5.3.

2.5.1. The best combination was a max depth of 9, ‘sqrt’ max features, and 50 trees, with a CV accuracy of 0.846. Depth had the largest effect, with accuracy rising from about 0.823 at depth 5 to 0.846 at depth 9, while the number of trees made little difference. The ‘sqrt’ and ‘log2’ settings scored identically because, with only 5 attributes, both round down to 2 attributes per split. Since the best depth sits at the edge of our grid, a deeper forest may perform even better.

2.5.2. The forest reached a test accuracy of 0.838, close to its CV score, indicating it generalizes well. This is above the 0.80 that could be reached by labeling every wine as not good, but as with any imbalanced dataset, accuracy alone does not show how well the forest identifies good wines.

2.6. Before constructing the Random Forest model, we loaded the Wine Quality dataset into a Pandas DataFrame and examined it for consistency, checking for null values, missing data, and any data inconsistencies (*Fig. 2.6.1*).

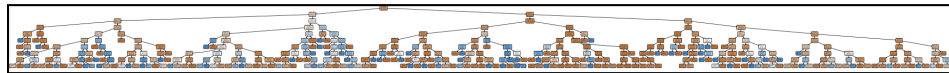


Fig. 2.6.1.

2.6.1. Unlike a single decision tree, this tree’s first split is on **density** rather than **alcohol**. Since each split only considers 2 randomly chosen attributes, alcohol was likely not available at the root, which shows how the forest’s randomization produces different, less correlated trees, **alcohol** still appears in the next level on both sides, and the only “Good” nodes shown come from low-density, high-alcohol wines.

2.6.2. The root node lists 3,240 samples but a value count of 5,197 wines. This reflects bootstrapping: each tree trains on a sample drawn with replacement, so some wines appear multiple times while others are left out. The split thresholds are also shown on the standardized scale, which is why many are negative.

2.7. To evaluate the tuned Random Forest on the test set, we examined its classification report and accuracy, confusion matrix, ROC and Precision-Recall curves, and feature importances (*Fig. 2.7.1 - 2.7.5*).

	precision	recall	f1-score	support
0	0.86	0.96	0.91	1044
1	0.68	0.34	0.46	256
accuracy			0.84	1300
macro avg	0.77	0.65	0.68	1300
weighted avg	0.82	0.84	0.82	1300
Accuracy:	0.8384615384615385			

Fig. 2.7.1.

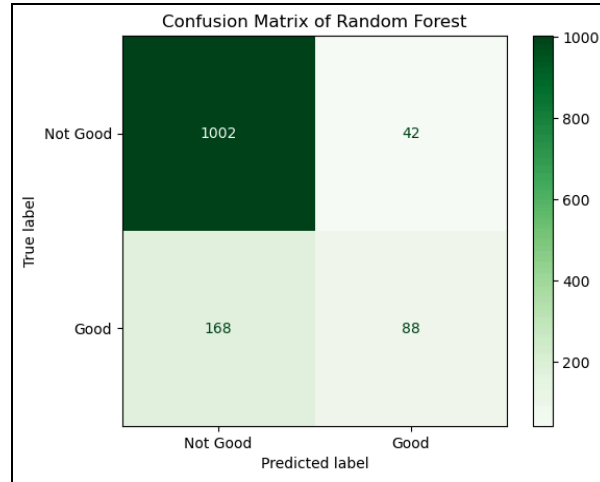


Fig. 2.7.2.

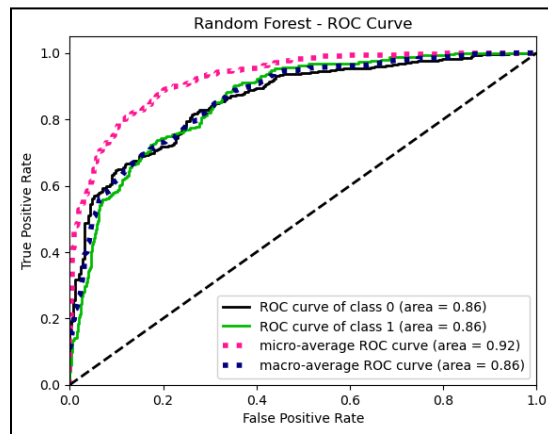


Fig. 2.7.3.

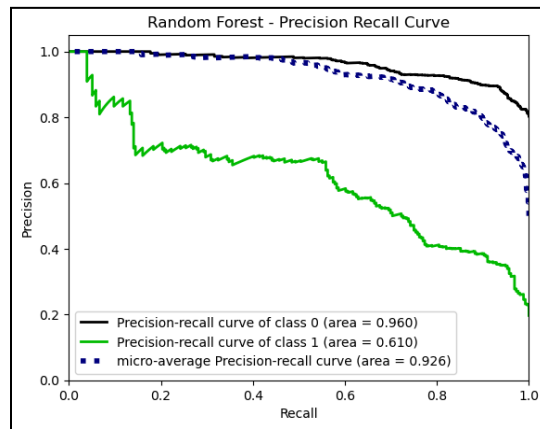


Fig. 2.7.4.

	alcohol	volatile_acidity	density	chlorides	citric_acid
0	0.318999	0.148714	0.238965	0.153183	0.140139

Fig. 2.7.5.

- 2.7.1. The classification report (*Fig. 2.7.1*) and confusion matrix (*Fig. 2.7.2.*) show the forest correctly identified 1,002 of 1,044 not-good wines and 88 of 256 good wines. For the Good class, this gives a precision of 0.68, meaning about two-thirds of its “Good” predictions are correct, but a recall of only 0.34, meaning it still misses roughly two out of three good wines, for an F1-score of 0.46. The Not Good classes score much higher (F1 of *0.91*), reflecting the 4-to-1 class imbalance. The forest’s overall accuracy of 0.838 is above the 0.80 reached by labeling every wine as not good, though most of that accuracy still comes from the majority class.
- 2.7.2. The ROC curve for the Good class has an area of 0.86 (*Fig. 2.7.3.*), showing the forest ranks good wines above not-good ones. Its curve is also smooth, since averaging many trees produces a wide range of probability values. The Precision-Recall curve gives a stricter view, with an area of 0.610 for the Good class compared with 0.960 for Not Good (*Fig. 2.7.4.*). Still, 0.610 is roughly three times the 0.20 a random guess would score, since only 256 out of 1,300 test wines are good, showing the forest captures meaningful patterns in the rarer class.
- 2.7.3. The feature importances are spread fairly evenly (*Fig. 2.7.5.*). **alcohol** leads at 0.319, followed closely by **density** (0.239), while **chlorides** (0.153), **volatile_acidity** (0.149), and **citric_acid** (0.140) contribute similar amounts. Density’s high importance reflects its strong overlap with alcohol: since each split only considers 2 random attributes often stands in for alcohol is unavailable, so the two share importance across the forest’s trees.

3. XGBoost Classifier

3.1. In order to simplify the confusion matrix and validate the model for this assignment, we must first change this problem to be a binary classifier. We have 7 classes ranked on quality, so we change the classes to instead be “good wine” and 1 or 0 where good is any wine ranked at a 7 or higher on a scale from 3-9. This is the only pre-processing to do as there are no null values.

3.2. After pre-processing we can train the model and do our hyper parameter tuning. In order to do this, we use GridSearchCV which will try all possible hyperparameters we give and then run cross validation on each of the models to find the best combination of hyperparameters. In this scenario we find that the best hyperparameters are listed in figure 4.2.1. “Eta” is also known as learning rate, this model runs a gradient descent to build its trees and the gradient descent is run to find the best weights, this learning rate is applied to the gradient descent. “Max depth” is the number of times the tree will split, how “tall” the tree is. And “N estimators” is the number of times the model is “boosted” which is how many weak models are created.

```
Fitting 5 folds for each of 27 candidates, totalling 135 fits  
0.8145305856576064  
{'eta': 0.3, 'max_depth': 2, 'n_estimators': 25}
```

Fig 3.2.1, Hyperparameter Tuning Results

3.3. After preprocessing and tuning the hyperparameters we are ready to evaluate our model. The model actually performs very poorly on this dataset traditionally. The target variable is “quality” which is a subjective target making the parameters only have correlation rather than being true indicators of what the quality actually is. The classes are also incredibly lopsided toward lower quality wines, so the model performs really well on those but very poorly on the higher quality because there just isn’t as much data.

	precision	recall	f1-score	support
0	0.8504	0.9667	0.9048	5220
1	0.6909	0.3046	0.4228	1277
accuracy			0.8365	6497
macro avg	0.7706	0.6356	0.6638	6497
weighted avg	0.8190	0.8365	0.8101	6497

Fig 3.3.1, Classification Report

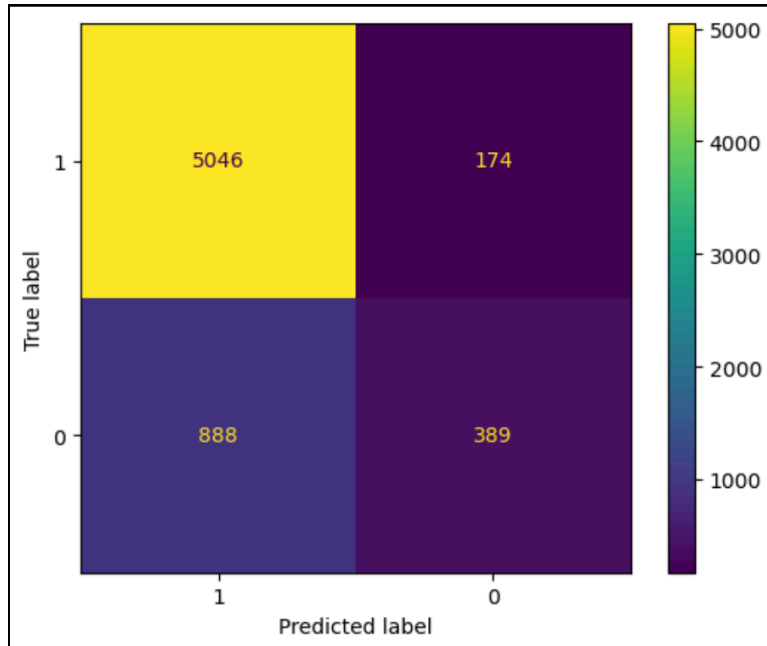


Fig 3.3.2, Confusion Matrix

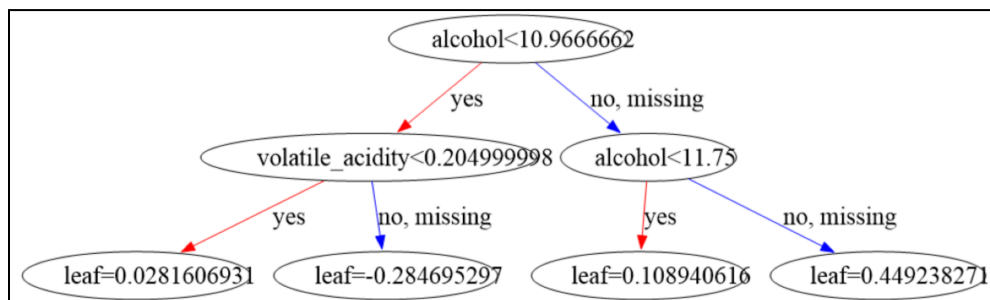


Fig 3.3.3, Tree Visualization

4. Adaboost Classifier

4.1. AdaBoosting is an extremely simple algorithm. It uses trees of depth 1 to split datasets and then do the classification on those datasets. Since this is also a boosting algorithm using trees, the model is very similar to XGBoost. All of the pre-processing is the same; it only differs once it gets to hyperparameter tuning - we also altered this dataset to be a binary classification.

4.2. For the hyperparameter tuning, AdaBoost is a very simple model so it only has 2 hyperparameters, those being “N estimators” and “Learning Rate.” After trying out various different combinations using GridSearchCV, the results I got are found in Figure 4.2.1.

```
0.8099117664475634  
{'learning_rate': 0.1, 'n_estimators': 200}
```

Fig 4.2.1, Hyperparameter Tuning

4.3. Again this dataset is highly subjective and has also been altered to be a binary classification problem. The classes are also very poorly distributed in the main dataset. This means that the models will not perform as well on this dataset as it would on others.

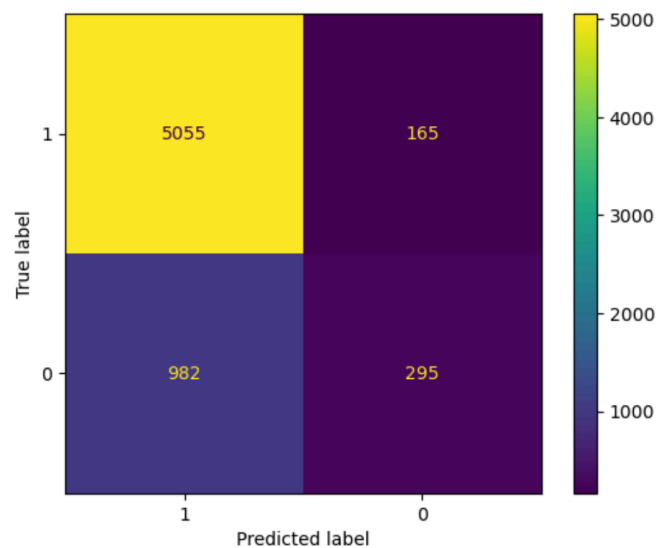


Fig 4.3.1, Confusion Matrix

	precision	recall	f1-score	support
0	0.8373	0.9684	0.8981	5220
1	0.6413	0.2310	0.3397	1277
accuracy			0.8235	6497
macro avg	0.7393	0.5997	0.6189	6497
weighted avg	0.7988	0.8235	0.7883	6497

Fig 4.3.2, Classification Report

5. Comparison of the Models

- 5.1. On this particular dataset, the model that performed the best is the Random Forest Classifier. That does make sense given the fact that the dataset is highly subjective, trying out random subsets of features will give you the best direction of what exactly to look at. AdaBoost and XGBoost will look at all the features every time. The subjectivity means that the person creating the dataset will have more favor to some of the features that the models don't notice at first. For most datasets boosting is going to be preferred as they learn off of each other while learning sequentially with each model getting smarter the more estimators it has. But some datasets will favor particular models and in this case Random Forest shined over some of the others.